Deklarativni operacijski sustavi
Seminar

Karlo Pušelj

Prirodoslovno-matematički fakultet — Matematički odsjek
Sveučilište u Zagrebu

9. listopada 2026.

Sadržaj

1. Uvod
2. Kratki izlet u prošlost
3. Upravitelji paketa
4. Nix, Nixpkgs i NixOS
5. Derivacije i determinizam
6. Praktični primjer

Motivacija

Što nakon kvara računala?

Imperativni i deklarativni pristup

Imperativni pristup	**Deklarativni pristup**
„Instaliraj SSH pa uključi servis.”	„SSH servis treba biti uključen.”
Pamtimo što smo napravili.	Zapisujemo što želimo imati.
Rezultat ovisi o početnom stanju i njegovim transformacijama.	Alat iz opisa priprema konfiguraciju.

Kako to izgleda u NixOS-u?

```
{
  services.openssh.enable = true;
}
```

- socijalni element: datoteku možemo čuvati u Git repozitoriju i dijeliti s drugima

Kratki izlet u prošlost

Kako su tržišni i licencni odnosi oblikovali razvoj softvera?

- konkurentski proizvođači UNIX-a razvijali su nekompatibilne varijante sustava $\rightarrow$ fragmentacija
- vlasničke licence ograničavaju dijeljenje i preraspodjelu softvera
- komercijalizacija sistemskog softvera narušava suradnju među programerima

Izvor: *The Open Group — povijest UNIX-a* [1]; *Richard Stallman — GNU Manifest (1985.)* [2]

Upravitelji paketa

Što je upravitelj paketa?

Što je upravitelj paketa?

Alat koji instalira, nadograđuje i uklanja softverske pakete te prati njihove ovisnosti.

```
aplikacija → biblioteka → druga biblioteka
```

- primjeri: **APT**, **DNF**, **pacman**
- paket sadrži datoteke programa i metapodatke
- ovisnosti opisuju što je programu potrebno za rad

Gdje tradicionalni model počne pucati?

- višestruke verzije paketa ne mogu koegzistirati $\rightarrow$ „DLL hell”
- nadogradnje nisu atomičke
- nedostatak predvidljivosti
- nedostatak pratljivosti
- gradnja paketa je ovisna o stanju sustava $\rightarrow$ skrivene ovisnosti
- problemi s ažuriranjem korisnički modificiranih komponenti

Izvor: *Dolstra i Löh — NixOS: A Purely Functional Linux Distribution (2008.)* [3]

Nix, Nixpkgs i NixOS

Naziv	Što označava?
Nix	Upravitelj paketa i sustav gradnje; također naziv jezika za opise.
Nixpkgs	Zbirka opisa paketa i biblioteka za rad s njima.
NixOS	Linux distribucija koja Nix koristi za izgradnju konfiguracije sustava.

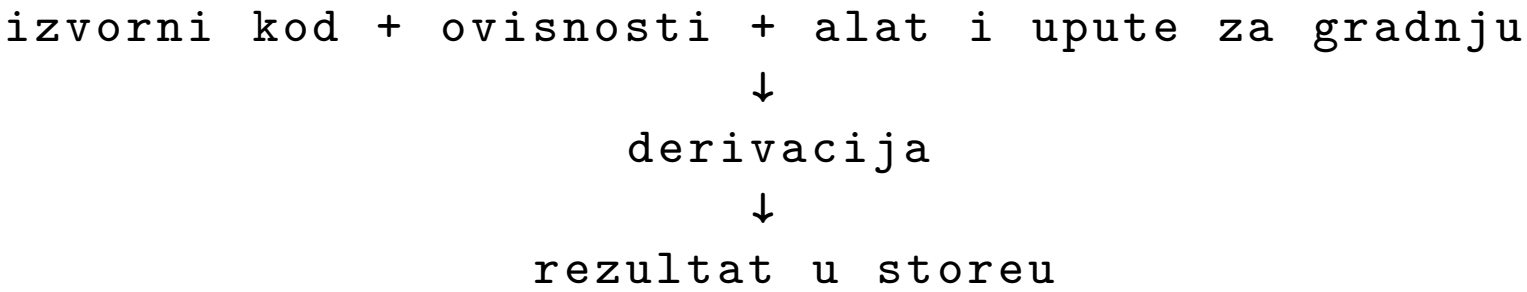
Temelj Nix-a: Nix store

```
/nix/store/<hash-A>-biblioteka-1.0/
/nix/store/<hash-B>-biblioteka-2.0/

program A → biblioteka-1.0
program B → biblioteka-2.0
```

- paketi se spremaju pod zasebne putanje u `/nix/store`
- postojeći rezultati gradnje ne mijenjaju se pri nadogradnji
- različite verzije i varijante mogu koegzistirati

Derivacije



- derivacija opisuje konkretan postupak gradnje i njegove ulaze
- ovisnosti zajedno tvore graf gradnje
- „binarna pričuvna memorija”

Postoji li ispravan redoslijed gradnje?

Modeliramo gradnju konačnim usmjerenim grafom $G = (V, E)$:

- vrh predstavlja korak gradnje
- brid $u \to v$ znači da gradnja v treba rezultat gradnje u

Teorem o topološkom poretku. Konačan usmjeren graf ima topološki poredak ako i samo ako nema usmjerenih ciklusa.
Topološki poredak je niz svih vrhova $v_1, \ldots, v_n$ takav da

$$v_i \to v_j \implies i < j.$$

- svaku ovisnost možemo izgraditi prije paketa koji je koristi
- poredak ne mora biti jedinstven

Izvor: *Cornell — topološki poredak* [4]

Skica dokaza

Dokazujemo indukcijom po broju vrhova:

1. prazan graf ima trivijalan poredak
2. svaki neprazan konačan aciklički graf ima vrh bez ulaznih bridova; u suprotnom bismo neprestanim praćenjem prethodnika zbog konačnosti ponovili vrh i dobili ciklus
3. odaberemo takav vrh v i uklonimo ga; preostali graf i dalje je acikličan pa po indukcijskoj pretpostavci ima topološki poredak
4. stavimo v na početak tog poretka jer nema ulaznih bridova, a svi njegovi izlazni bridovi vode prema kasnijim vrhovima

Obratno, ciklus bi zahtijevao da se njegov početni vrh nalazi prije samoga sebe, što je nemoguće. □

Malo o Nix jeziku

```
a: a + 1 # lambda funkcija
a: b: a + b # lambda funkcija koja vraća lambda funkciju
{a = 1; b = 1} # atributni skup
{a, b}: a + b # lambda funkcija koja prima atributni skup
```

Malo o Nix jeziku (nastavak)

```
{pkgs, nazivPosluzitelja, ...}: {
  networking.hostName = nazivPosluzitelja;
  services.openssh.enable = true;
  environment.systemPackages = [ ];
}
```

- `{ ...}`: skup atributa, odnosno parova ime–vrijednost
- `"seminar"` je tekst, a `true` logička vrijednost
- `[ ...]` je lista; elementi se odvajaju razmacima
- točke označavaju ugniježđene atribute; dodjelu završava `;`

Primjer derivacije iz Nixpkgs

DEMO *pokazati `src` direktno iz Nixpkgs*

Determinizam derivacija

Što kontroliramo?

- verziju paketa, izvore i ostale ulaze
- platformu za koju gradimo
- okruženje gradnje $\rightarrow$ pješčanik

Za raspravu

Kada derivacija ne bi bila deterministička?

Praktični primjer: jednostavni server

```
{ config, pkgs, ... }:

{
  # SSH pristup
  services.openssh.enable = true;

  # Vatrozid propušta samo SSH
  networking.firewall.allowedTCPPorts = [ 22 ];

  # Osnovni korisnik
  users.users.server.isNormalUser = true;
}
```

Generacije konfiguracija

```
generacija 1 → generacija 2 → generacija 3
konfiguracija 1 → konfiguracija 2 → konfiguracija 3
```

DEMO *demonstracija generacija*

- nova se konfiguracija prvo izgradi
- prebacivanje profila na novu generaciju je atomično
- stara generacija ostaje dostupna dok je ne uklonimo

Kako se izgradi sustav

```
sudo nixos-rebuild switch
```

opis → provjera i evaluacija → gradnja/preuzimanje → aktivacija

Naredba `nixos-rebuild ...`	**Rezultat**
`build`	Izgradi konfiguraciju, bez aktivacije.
`test`	Aktivira je, bez postavljanja kao zadane za pokretanje.
`boot`	Postavi je za sljedeće pokretanje, bez aktivacije sada.
`switch`	Aktivira je i postavi za sljedeće pokretanje.

Što ako nova konfiguracija ne radi?

```
sudo nixos-rebuild switch --rollback
```

- u pokrenutom sustavu možemo se vratiti na prethodnu generaciju
- ako se sustav ne pokreće, biramo sačuvanu generaciju u izborniku za pokretanje
- povrat konfiguracije ne vraća osobne datoteke, no njihov se povrat može riješiti pomoću BTRFS-a

Izvor: *povrat NixOS konfiguracije* [5]

Reproducibilnost

Hoće li konfiguracija generirati isti sustav danas i za mjesec dana?

Pahuljice (eng. *flake*) spašavaju

Datoteka	**Uloga u projektu s flakes**
`configuration.nix`	NixOS postavke.
`flake.nix`	Deklarira ulaze i izlaze projekta.
`flake.lock`	Bilježi zaključane revizije i hashove ulaza.

- za ponavljanje konfiguracije dijelimo i zaključane ulaze
- nadogradnja ulaza je namjerna promjena koju možemo pregledati

Korist i bez prelaska na NixOS

Primjer datoteke `shell.nix`:

```
{ pkgs ? import <nixpkgs> { } }:
pkgs.mkShell {
  packages = [ pkgs.git pkgs.curl ];
}
```

```
nix-shell
```

- dobivamo razvojnu ljusku s navedenim alatima
- izlaskom iz ljuske prestajemo koristiti to okruženje
- `<nixpkgs>` ovdje koristi lokalno odabranu zbirku; primjer **ne zaključava verzije**

Praktični primjeri

DEMO *pokazati nekoliko praktičnih primjera*

Zašto se Nix/NixOS ipak isplati?

- zajednička konfiguracija za više računala
- sustav otvorenog koda s najširom podrškom i najvećim brojem paketa
- rad s cijelim operacijskim sustavom i paketima bez glavobolja
- čist sustav bez naknadnog smeća ako imate OKP
- praktičan rad s računalom
- lakša suradnja i dijeljenje

Pitanja i daljnje čitanje

Koji biste dio svojeg računala prvi opisali deklarativno?

- `https://nix.dev/` — vodiči i primjeri [6]
- `https://nixos.org/manual/nixos/stable/` — NixOS priručnik [7]
- `https://search.nixos.org/options` — pretraživanje NixOS opcija [8]
- `https://search.nixos.org/packages` — pretraživanje paketa [9]

Literatura I

[1] T. O. Group. *History of UNIX*. URL: https://www.unix.org/unix_history.html (pogledano 9. 10. 2026.).

[2] R. Stallman. *The GNU Manifesto*. 1985. URL: https://www.gnu.org/gnu/manifesto.en.html (pogledano 9. 10. 2026.).

[3] E. Dolstra i A. Löh. „NixOS: A Purely Functional Linux Distribution". *Proceedings of the 13th ACM SIGPLAN International Conference on Functional Programming*. Association for Computing Machinery, 2008., str. 367–378. URL: https://nixos.org/~eelco/pubs/nixos-icfp2008-final.pdf.

[4] C. University. *Graph Representations and Traversals*. CS 3110, Lecture 22. 2009. URL: https://www.cs.cornell.edu/courses/cs3110/2009sp/lectures/lec22.html (pogledano 9. 10. 2026.).

Literatura II

[5] *NixOS Manual: Rolling Back*. URL: https://nixos.org/manual/nixos/stable/#sec-rollback (pogledano 9. 10. 2026.).

[6] *nix.dev*. URL: https://nix.dev/ (pogledano 9. 10. 2026.).

[7] *NixOS Manual*. URL: https://nixos.org/manual/nixos/stable/ (pogledano 9. 10. 2026.).

[8] *NixOS Search: Options*. URL: https://search.nixos.org/options (pogledano 9. 10. 2026.).

[9] *NixOS Search: Packages*. URL: https://search.nixos.org/packages (pogledano 9. 10. 2026.).